# HOME BUDGET

## Documento de requisitos técnicos e regras de negócio

Objetivo
Definir o escopo inicial de uma aplicação de orçamento doméstico para registrar contas a pagar, receitas salariais, pagamentos e recebimentos e apresentar uma visão mensal do orçamento.

Versão: 1.0
Data: 17 de outubro de 2026
Escopo: backend REST em Java/Spring Boot, PostgreSQL e documentação da API.

Este documento especifica o comportamento esperado. Recursos descritos como planejados precisam ser confirmados durante a implementação.

# 1. Visão geral

O Home Budget ajuda a organizar o orçamento doméstico, acompanhar vencimentos e pagamentos e distinguir saldo projetado de saldo realizado em um mês.

## 1.1 Objetivos

- Centralizar o cadastro de contas e receitas mensais.
- Separar despesas pelo período de recebimento que deve cobri-las.
- Registrar pagamentos integrais ou parciais e recebimentos.
- Apresentar indicadores mensais claros e consistentes.
- Preservar o histórico de registros, inclusive contas canceladas.

## 1.2 Escopo da primeira versão

- API REST para contas e receitas.
- PostgreSQL com migrations Flyway.
- Validação de entradas, tratamento padronizado de erros e testes automatizados.
- Resumo financeiro mensal.
- Documentação OpenAPI/Swagger e instruções de execução.

Fora do escopo inicial: autenticação multiusuário, integração bancária, notificações, recorrência automática e frontend React. Podem ser considerados em versões futuras.

# 2. Requisitos funcionais

| ID | Requisito | Descrição |
|---|---|---|
| RF-01 | Cadastrar conta | Registrar nome, valor total, vencimento e categoria. |
| RF-02 | Consultar contas | Listar contas e consultar uma por identificador. |
| RF-03 | Filtrar contas | Filtrar por mês, status, categoria e período financeiro. |
| RF-04 | Atualizar conta | Alterar dados editáveis respeitando pagamentos já registrados. |
| RF-05 | Cancelar conta | Preservar o histórico e excluir contas canceladas dos totais. |
| RF-06 | Registrar pagamento | Registrar pagamento total ou parcial e atualizar saldo e status. |
| RF-07 | Cadastrar receita | Registrar descrição, valor, tipo e data prevista. |
| RF-08 | Consultar receitas | Listar e consultar receitas por mês e status. |
| RF-09 | Registrar recebimento | Marcar receita recebida e informar data real. |
| RF-10 | Resumo mensal | Exibir receitas, despesas e saldos projetado/realizado. |
| RF-11 | Agrupar por período | Consultar contas associadas ao recebimento do quinto dia útil ou dia 20. |
| RF-12 | Documentar API | Disponibilizar documentação e exemplos de payloads. |

# 3. Regras de negócio

## 3.1 Valores monetários

- RN-01: usar BigDecimal no Java e NUMERIC(12,2) no PostgreSQL como padrão inicial.
- RN-02: o valor total de uma conta deve ser maior que zero.
- RN-03: o valor pago não pode ser negativo nem ultrapassar o valor total.
- RN-04: evitar float/double em cálculos financeiros.

## 3.2 Categorias de conta

| Enum | Descrição |
|---|---|
| WATER | Água |
| ELECTRICITY | Energia elétrica |
| INTERNET | Internet |
| HOUSING | Aluguel, condomínio e despesas habitacionais |
| FOOD | Mercado e alimentação |
| TRANSPORT | Combustível e transporte |
| HEALTH | Saúde e medicamentos |
| EDUCATION | Educação e cursos |
| SUBSCRIPTIONS | Assinaturas e serviços recorrentes |
| OTHER | Outras despesas |

Persistir enums como texto (EnumType.STRING). Se houver CHECK no banco, novas categorias exigem migration para atualizar a restrição.

## 3.3 Status de conta

| Status | Significado | Regra |
|---|---|---|
| PENDING | Pendente | Ainda não quitada integralmente; pode ter pagamento parcial. |
| PAID | Paga | Totalmente quitada. |
| CANCELLED | Cancelada | Mantida no histórico, excluída dos totais. |

- RN-05: uma conta nova começa como PENDING.
- RN-06: pagamento maior que zero e menor que o total mantém PENDING.
- RN-07: ao quitar o total, o status passa a PAID.
- RN-08: conta CANCELLED não aceita pagamentos nem compõe totais financeiros.
- RN-09: validar transições para impedir inconsistência entre status e valores.

## 3.4 Pagamentos

- RN-10: saldo em aberto = valor total − valor pago acumulado.
- RN-11: pagamento parcial não equivale a quitação integral.
- RN-12: paidAt representa a quitação final no modelo simples; múltiplos pagamentos em datas diferentes pedem uma entidade Payment com histórico próprio.
- RN-13: o total pago não pode ultrapassar o valor da conta.

Decisão: o modelo atual com paidAmount e paidAt serve para um registro acumulado simples. Para auditoria de cada parcela, evoluir para tabela própria de pagamentos.

## 3.5 Períodos financeiros

| Vencimento | Período | Interpretação |
|---|---|---|
| Dias 1–19 | FIFTH_BUSINESS_DAY | Despesa alocada ao recebimento do quinto dia útil. |
| Dia 20 até o fim do mês | DAY_20 | Despesa alocada ao recebimento do dia 20. |

- RN-14: derivar o período do dia de dueDate; não é necessário persistir o período se ele puder ser calculado.
- RN-15: filtrar pelo mês e ano de dueDate; período financeiro não substitui o mês de referência.
- RN-16: dia 19 pertence ao primeiro período; dia 20 pertence ao segundo.
- RN-17: se o sistema calcular o quinto dia útil, considerar fins de semana e feriados aplicáveis. Na primeira versão, expectedDate da receita pode ser informada explicitamente.

## 3.6 Receitas

| Campo | Descrição |
|---|---|
| description | Descrição, por exemplo salário. |
| amount | Valor esperado da receita; maior que zero. |
| type | FIFTH_BUSINESS_DAY ou DAY_20. |
| expectedDate | Data prevista de recebimento. |
| status | EXPECTED ou RECEIVED. |
| receivedAt | Data efetiva de recebimento, preenchida quando recebida. |

- RN-18: uma receita nova começa como EXPECTED.
- RN-19: ao registrar o recebimento, passar para RECEIVED e registrar a data real.
- RN-20: contabilizar cada receita uma única vez no resumo.
- RN-21: receita esperada não deve ser tratada como dinheiro efetivamente recebido.

## 3.7 Resumo mensal

O resumo recebe um mês de referência, preferencialmente YYYY-MM. Os indicadores precisam ter semântica documentada e consistente.

| Indicador | Definição proposta |
|---|---|
| expectedIncome | Total originalmente esperado de receitas para o mês, inclusive as já recebidas. |
| receivedIncome | Receitas efetivamente recebidas, segundo a regra de data adotada. |
| totalExpenses | Valor total das contas não canceladas com vencimento no mês. |
| paidExpenses | Valores efetivamente pagos nas contas não canceladas com vencimento no mês. |
| pendingExpenses | Saldo em aberto das contas não canceladas. |
| projectedBalance | Receitas esperadas menos despesas totais previstas. |
| realizedBalance | Receitas recebidas menos valores efetivamente pagos, segundo a regra de data adotada. |

Definição importante: saldo realizado por mês de vencimento não é necessariamente fluxo de caixa real. Para fluxo de caixa, agrupar receitas por receivedAt e pagamentos pela data efetiva do pagamento. pendingExpenses deve somar amount − paidAmount, tratando paidAmount nulo como zero. Em condições válidas, paidExpenses + pendingExpenses = totalExpenses.

# 4. Requisitos não funcionais

| ID | Área | Requisito |
|---|---|---|
| RNF-01 | Plataforma | Java 21 como versão-alvo inicial. |
| RNF-02 | Framework | Spring Boot, Spring Web e Spring Data JPA. |
| RNF-03 | Banco | PostgreSQL. |
| RNF-04 | Schema | Flyway; evitar atualização automática de schema em produção. |
| RNF-05 | API | REST com prefixo /api/v1. |
| RNF-06 | Validação | Bean Validation nos DTOs e validações de negócio nos serviços. |
| RNF-07 | Erros | Respostas consistentes sem expor stack traces. |
| RNF-08 | Testes | JUnit 5, Mockito e testes de integração dos fluxos críticos. |
| RNF-09 | Documentação | OpenAPI/Swagger e README. |
| RNF-10 | Segurança de configuração | Segredos em variáveis de ambiente/configuração externa, nunca no código versionado. |
| RNF-11 | Execução local | Docker Compose para facilitar a reprodução do PostgreSQL. |
| RNF-12 | Manutenibilidade | Separar controllers, DTOs, serviços/casos de uso e persistência sem camadas desnecessárias. |

# 5. Modelo de dados inicial

## 5.1 Bill — contas

| Campo | Tipo sugerido | Observação |
|---|---|---|
| id | BIGINT identity | Chave primária. |
| name | VARCHAR(100) | Obrigatório. |
| amount | NUMERIC(12,2) | Obrigatório; maior que zero. |
| due_date | DATE | Obrigatório. |
| bill_category | VARCHAR(30) | Enum textual. |
| bill_status | VARCHAR(30) | Enum textual. |
| paid_amount | NUMERIC(12,2) | Definir convenção para zero/nulo. |
| paid_at | DATE | Rever se houver pagamentos parciais múltiplos. |
| created_at | TIMESTAMP WITH TIME ZONE | Preenchido na criação. |

## 5.2 Income — receitas

| Campo | Tipo sugerido | Observação |
|---|---|---|
| id | BIGINT identity | Chave primária. |
| description | VARCHAR(100) | Obrigatório. |
| amount | NUMERIC(12,2) | Obrigatório; maior que zero. |
| type | VARCHAR(30) | FIFTH_BUSINESS_DAY ou DAY_20. |
| expected_date | DATE | Data prevista. |
| status | VARCHAR(20) | EXPECTED ou RECEIVED. |
| received_at | DATE | Data real, após recebimento. |
| created_at | TIMESTAMP WITH TIME ZONE | Recomendado para auditoria. |

Tipos e nomes são propostas. As migrations devem corresponder às entidades e regras efetivamente implementadas.

# 6. API REST proposta

| Método e rota | Objetivo |
|---|---|
| POST /api/v1/bills | Cadastrar conta. |
| GET /api/v1/bills | Listar/filtrar contas por mês, status, categoria ou período. |
| GET /api/v1/bills/{id} | Consultar conta por ID. |
| PUT /api/v1/bills/{id} | Atualizar dados editáveis. |
| PATCH /api/v1/bills/{id}/pay | Registrar pagamento total ou parcial. |
| PATCH /api/v1/bills/{id}/cancel | Cancelar conta. |
| POST /api/v1/incomes | Cadastrar receita. |
| GET /api/v1/incomes | Listar/filtrar receitas. |
| GET /api/v1/incomes/{id} | Consultar receita por ID. |
| PATCH /api/v1/incomes/{id}/receive | Registrar recebimento. |
| GET /api/v1/budget/summary?month=YYYY-MM | Consultar resumo mensal. |

Rotas propostas para orientar o desenho da API; não indicam que já estejam implementadas.

# 7. Validação e erros

- 400 Bad Request para payload inválido ou parâmetros malformados.
- 404 Not Found quando o recurso não existir.
- 409 Conflict para operação incompatível com o estado atual, quando apropriado.
- Rejeitar valores de enum desconhecidos.
- Validar valores, datas e transições na camada de negócio além dos DTOs.
- Usar tratamento centralizado de exceções para manter respostas consistentes.

# 8. Estratégia de testes

| Grupo | Casos essenciais |
|---|---|
| Contas | Criação válida; valor zero/negativo; categoria inválida; consulta inexistente. |
| Pagamentos | Quitação integral; parcial; excesso; conta cancelada; transição para PAID. |
| Períodos | Vencimentos nos dias 1, 19, 20 e último dia do mês. |
| Receitas | Cadastro EXPECTED; recebimento; consistência entre datas e status. |
| Resumo | Sem registros; contas pagas/pendentes/canceladas; receitas esperadas/recebidas; totais consistentes. |
| Integração | Persistência PostgreSQL, migrations Flyway e respostas HTTP. |

# 9. Ordem de implementação sugerida

- Etapa 1 — Fundação: projeto, PostgreSQL, Docker Compose e Flyway.
- Etapa 2 — Contas: entidade, DTOs, persistência, CRUD e cancelamento.
- Etapa 3 — Pagamentos: pagamentos totais/parciais e testes de consistência.
- Etapa 4 — Receitas: cadastro, consulta e registro de recebimento.
- Etapa 5 — Regras financeiras: período e resumo mensal.
- Etapa 6 — Qualidade: integração, OpenAPI, README e revisão das regras.

# 10. Decisões em aberto

- Definir se paidAmount será sempre zero em vez de NULL.

- Definir se pagamentos parciais terão histórico próprio na primeira versão.
- Definir se o saldo realizado será agrupado por mês de vencimento/previsto ou pelas datas efetivas de entrada e saída.
- Definir calendário de feriados caso a data do quinto dia útil seja calculada automaticamente.
- Avaliar recorrência automática de contas em versão posterior.
- Definir autenticação e suporte a múltiplos usuários quando houver frontend.

# 11. Critério de conclusão da primeira versão

A primeira versão estará concluída quando os fluxos principais de contas, pagamentos, receitas, recebimentos e resumo mensal estiverem implementados; as regras de negócio estiverem cobertas por testes; as migrations puderem ser aplicadas em banco limpo; e a API puder ser executada localmente seguindo o README.

Fim do documento.